



南開大學
Nankai University

计算机学院
人工智能导论实验报告

实验四：特征人脸识别

姓名：林盛森

学号：2312631

专业：计算机科学与技术

2025 年 5 月 9 日

目录

1 问题重述	2
1.1 实验背景	2
1.2 实验内容	2
1.3 实验要求	2
2 设计思想	2
3 代码内容	2
3.1 数据预处理	2
3.2 展示图片函数	3
3.3 特征人脸算法	4
3.4 人脸识别模型（投影压缩）	5
3.5 人脸重建模型	7
4 实验结果	10
4.1 样例测试	10
5 总结	10

1 问题重述

1.1 实验背景

本实验采用特征脸 (Eigenface) 算法进行人脸识别。

特征脸 (eigenface) 是第一种有效的人脸识别方法, 通过在一大组描述不同人脸的图像上进行主成分分析 (PCA) 获得。本次实验要求大家构建一个自己的人脸库 (建议): 大家可以选择基于 ORL 人脸库添加自己搜集到的人脸图像形成一个更大的人脸库, 要求人脸库中的每一张图像都只包含一张人脸且眼睛的中心位置对齐 (通过裁剪或缩放, 使得每张人脸图像大小尺寸一致且人脸眼睛的中心位置对齐)。为了方便同学们操作, 大家也可以选择直接基于 ORL 人脸库进行本次实验。

1.2 实验内容

在模型训练过程中, 首先要根据测试数据求出平均脸, 然后将前 K 个特征脸保存下来, 利用这 K 个特征脸对测试人脸进行识别, 此外对于任意给定的一张人脸图像, 可以使用这 K 个特征脸对原图进行重建。

1.3 实验要求

1. 求解人脸图像的特征值与特征向量构建特征脸模型
2. 利用特征脸模型进行人脸识别和重建, 比较使用不同数量特征脸的识别与重建效果
3. 使用 Python 语言

2 设计思想

核心部分是使用主成分分析获得特征向量继而获得特征脸向量, 实现降维, 利用特征脸向量可以表示输入的人脸图像 (线性组合), 从而进行人脸识别分析, 只需比较线性组合的结果的相似性即可完成人脸识别。具体设计思想在代码分析中给出。

3 代码内容

3.1 数据预处理

```
1 # 导入必要的包
2 import matplotlib.pyplot as plt
3 import numpy as np
4 import cv2
5 from PIL import Image
6 import os
7
8 def split_data(nPerson, nPicture, data, label):
9     """
10     分割数据集
11
12     :param nPerson : 志愿者数量
13     :param nPicture: 各志愿者选入训练集的照片数量
```

```

14 :param data : 等待分割的数据集
15 :param label: 对应数据集的标签
16 :return: 训练集, 训练集标签, 测试集, 测试集标签
17 """
18 # 数据集大小和意义
19 allPerson, allPicture, rows, cols = data.shape
20
21 # 划分训练集和测试集
22 train = data[:nPerson,:nPicture,:].reshape(nPerson*nPicture, rows*cols)
23 train_label = label[:nPerson, :nPicture].reshape(nPerson * nPicture)
24 test = data[:nPerson, nPicture:, :, :].reshape(nPerson*(allPicture - nPicture),
25         rows*cols)
26 test_label = label[:nPerson, nPicture:].reshape(nPerson * (allPicture - nPicture))
27
28 # 返回: 训练集, 训练集标签, 测试集, 测试集标签
29 return train, train_label, test, test_label

```

这一段我们导入了需要使用的包, matplotlib.pyplot 这个包用于展示人脸图像、特征脸等, numpy 用于矩阵操作和向量化操作, cv2 可以用于图像预处理, 在计算机视觉和图像处理方面应用, PIL 用于图像处理, os 提供操作系统接口。

然后定义了 split_data 这个函数, 用于划分训练集和测试集, 并且在其中, 对于输入数据 data 和 label 进行了展平操作, 将每个 rows*cols 的图像转换为 1* (rows*cols) 的行向量, 减少复杂性和计算量。

3.2 展示图片函数

```

1 def plot_gallery(images, titles, n_row=3, n_col=5, h=112, w=92): # 3行4列
2     """
3     展示多张图片
4
5     :param images: numpy array 格式的图片
6     :param titles: 图片标题
7     :param h: 图像reshape的高
8     :param w: 图像reshape的宽
9     :param n_row: 展示行数
10    :param n_col: 展示列数
11    :return:
12    """
13    # 展示图片
14    plt.figure(figsize=(1.8 * n_col, 2.4 * n_row))
15    plt.subplots_adjust(bottom=0, left=.01, right=.99, top=.90, hspace=.35)
16    for i in range(n_row * n_col):
17        plt.subplot(n_row, n_col, i + 1)
18        plt.imshow(images[i].reshape((h, w)), cmap=plt.cm.gray)
19        plt.title(titles[i], size=12)
20        plt.xticks(())
21        plt.yticks(())
22    plt.show()

```

这段定义了 `plot_gallery` 这个函数，用于展示图片，通过使用 `matplotlib.pyplot`，可以一次性展示多张图片。

```

1 datapath = './ORL.npz'
2 ORL = np.load(datapath)
3 data = ORL['data']
4 label = ORL['label']
5 num_eigenface = 200
6
7 train_vectors, train_labels, test_vectors, test_labels = split_data(40, 5, data, label)
8 train_vectors = train_vectors / 255
9 test_vectors = test_vectors / 255
10
11 print("训练数据集:", train_vectors.shape)
12 print("测试数据集:", test_vectors.shape)

```

然后在这里加载数据，并调用了 `split_data` 这个函数，完成训练集和测试集的划分，训练集样本数为 200，测试集样本数为 80，每个图像样本的维度都为 1×10304 。

值得一提的是，这里进行了数值归一化操作，将像素值从整数范围 $[0, 255]$ 转换到浮点数范围 $[0, 1]$ ，提高了数值稳定性，使后续计算（如特征提取、相似度计算）更加稳定，并且有助于特征脸算法更快收敛。

3.3 特征人脸算法

```

1 def eigen_train(trainset, k=20):
2     """
3     训练特征脸 (eigenface) 算法的实现
4
5     :param trainset: 使用 get_images 函数得到的处理好的人脸数据训练集
6     :param K: 希望提取的主特征数
7     :return: 训练数据的平均脸，特征脸向量，中心化训练数据
8     """
9     #1. 平均人脸
10    avg_img = np.mean(trainset, axis=0)
11
12    #2. 特征向量 (使用主成分分析)
13    norm_img = trainset - avg_img # 中心化
14
15    n_samples = norm_img.shape[0]
16    S = np.dot(norm_img, norm_img.T) # 协方差矩阵 (奇异值分解) 维度更小
17
18    feature_values, feature_vectors = np.linalg.eigh(S) # 特征值、特征向量
19
20    feature_vectors = np.dot(norm_img.T, feature_vectors) # 转换到原始空间
21    sorted_indices = np.argsort(feature_values)[::-1]
22    feature_vectors = feature_vectors[:, sorted_indices[:k]]

```

```

23 #3.特征脸
24 feature = feature_vectors / np.linalg.norm(feature_vectors, axis=0)
25 feature=feature.T
26
27 # 返回：平均人脸、特征人脸、中心化人脸
28 return avg_img, feature, norm_img

```

然后这里是算法核心部分，用于获得特征脸向量。首先使用 `np.mean` 求取平均人脸，`axis=0` 表示沿垂直方向进行操作。接着需要使用主成分分析进行向量降维（采用了奇异值分解），先让每个图像向量减去平均向量获得中心化向量，然后获得协方差矩阵，`np.dot` 用于矩阵相乘，通过奇异值分解，这里计算的是 $\varphi \varphi^T$ 的特征向量和特征值，之后再左乘中心化向量转换到原始空间，即求出 $\varphi^T \varphi$ 的特征向量，通过这种做法，当我们要求取特征向量的时候，所求的是一个 $n \times n$ 的矩阵，而不是维度（`rows*cols`）*（`rows*cols`）的矩阵，这种做法对于求取小样本的特征向量有很大的作用，减少计算量和时间。

```

1 # 返回平均人脸、特征人脸、中心化人脸
2 avg_img, eigenface_vects, trainset_vects = eigen_train(train_vectors, num_eigenface)
3 print(eigenface_vects.shape)
4 # 打印两张特征人脸作为展示
5 eigenfaces = eigenface_vects.reshape((num_eigenface, 112, 92))
6 eigenface_titles = ["eigenface %d" % i for i in range(eigenfaces.shape[0])]
7 plot_gallery(eigenfaces, eigenface_titles, n_row=1, n_col=2)

```

这里调用了 `eigen_train` 这个函数，获得训练集的平均人脸、特征人脸和中心化人脸，并打印了两张特征人脸作为展示。之后我们就可以使用这些特征脸向量线性组合去表示输入的图像数据。

(200, 10304)

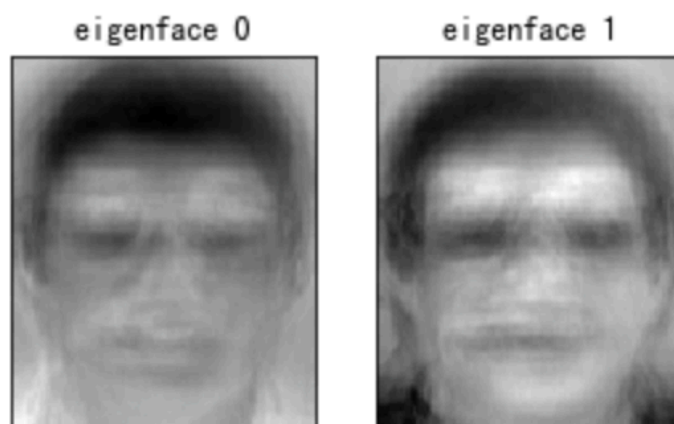


图 3.1: 特征人脸展示图

3.4 人脸识别模型（投影压缩）

```

1 def rep_face(image, avg_img, eigenface_vects, numComponents = 0):
2     """
3     用特征脸 (eigenface) 算法对输入数据进行投影映射，得到使用特征脸向量表示的数据
4     :param image: 输入数据
5     :param avg_img: 训练集的平均人脸数据

```

```

6 :param eigenface_vects: 特征脸向量
7 :param numComponents: 选用的特征脸数量
8 :return: 输入数据的特征向量表示, 最终使用的特征脸数量
9 """
10 numEigenFaces=numComponents
11 use_face=eigenface_vects[0:numEigenFaces]
12 representation=np.dot(image-avg_img, use_face.T) #利用特征脸去表示输入数据
13
14 # 返回: 输入数据的特征向量表示, 特征脸使用数量
15 return representation, numEigenFaces

```

这里是用于获取输入数据的特征脸表示, 特征向量 = (原始图像 - 平均脸) * 特征脸矩阵.T。

```

1 train_reps = []
2 for img in train_vectors:
3     train_rep, _ = rep_face(img, avg_img, eigenface_vects, num_eigenface)
4     train_reps.append(train_rep)
5
6 num = 0
7 for idx, image in enumerate(test_vectors):
8     label = test_labels[idx]
9     test_rep, _ = rep_face(image, avg_img, eigenface_vects, num_eigenface)
10
11     results = []
12     for train_rep in train_reps:
13         similarity = np.sum(np.square(train_rep - test_rep))
14         results.append(similarity)
15     results = np.array(results)
16
17     if label == np.argmin(results) // 5 + 1:
18         num = num + 1
19
20 print("人脸识别准确率: {}".format(num / 80 * 100))

```

这里就是人脸识别的核心, 先遍历训练图像, 将每张训练图像投影到特征脸空间, 得到其特征表示, 接着遍历每张测试图像, 计算测试图像在特征脸空间的表示, 通过计算测试图像与所有训练图像的相似度 (欧氏距离平方), 找到最相似的训练图像, 如果识别结果与真实标签匹配, 则计数器加 1。最后获得人脸识别准确率。

```

    if label == np.argmin(results) // 5 + 1:
        num = num + 1

print("人脸识别准确率: {}".format(num / 80 * 100))

```

人脸识别准确率: 91.25%

图 3.2: 人脸识别准确率

可以看到，人脸识别准确率为 91.25%。

3.5 人脸重建模型

```

1 def recFace(representations, avg_img, eigenVectors, numComponents, sz=(112,92)):
2     """
3     利用特征人脸重建原始人脸
4
5     :param representations: 表征数据
6     :param avg_img: 训练集的平均人脸数据
7     :param eigenface_vects: 特征脸向量
8     :param numComponents: 选用的特征脸数量
9     :param sz: 原始图片大小
10    :return: 重建人脸, str 使用的特征人脸数量
11    """
12    use_face=eigenVectors[0:numComponents]
13    face=np.dot(representations,use_face)+avg_img
14    # 返回: 重建人脸, str 使用的特征人脸数量
15    return face, 'numEigenFaces_{}'.format(numComponents)

```

这里定义了 recFace 函数，可以对压缩到特征脸空间的图像进行重建，获得原始图像，只需进行逆运算，重构图像 = 平均脸 + 特征向量 * 特征脸矩阵。

```

1 print("重建训练集人脸")
2 # 读取train数据
3 image = train_vectors[100]
4
5 faces = []
6 names = []
7 # 选用不同数量的特征人脸重建人脸
8 for i in range(20, 200, 20):
9     representations, numEigenFaces = rep_face(image, avg_img, eigenface_vects, i)
10    face, name = recFace(representations, avg_img, eigenface_vects, numEigenFaces)
11    faces.append(face)
12    names.append(name)
13
14 plot_gallery(faces, names, n_row=3, n_col=3)
15
16 print("-"*55)
17 print("重建测试集人脸")
18 # 读取test数据
19 image = test_vectors[54]
20
21 faces = []
22 names = []
23 # 选用不同数量的特征人脸重建人脸
24 for i in range(20, 200, 20):
25     representations, numEigenFaces = rep_face(image, avg_img, eigenface_vects, i)
26    face, name = recFace(representations, avg_img, eigenface_vects, numEigenFaces)

```



```
27 faces.append(face)
28 names.append(name)
29
30 plot_gallery(faces, names, n_row=3, n_col=3)
```

这里分别调用函数对训练集和测试集的映射到特征脸空间的特征向量（压缩）进行重建，选用不同数量的特征人脸重建人脸，获取原始图像。

重建训练集人脸

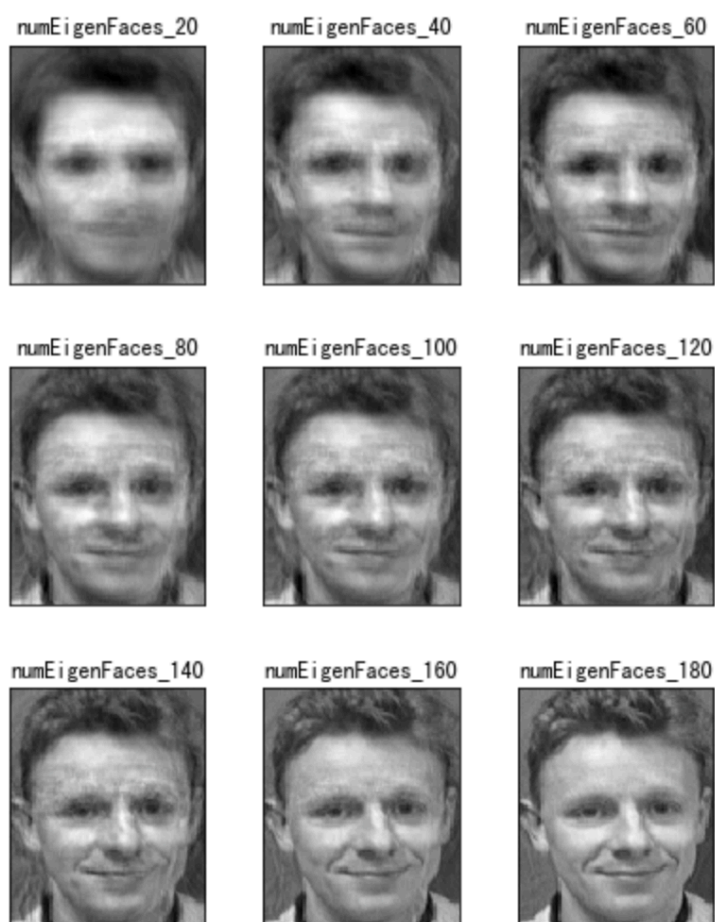


图 3.3: 训练集样例

重建测试集人脸

numEigenFaces_20



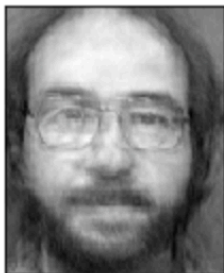
numEigenFaces_40



numEigenFaces_60



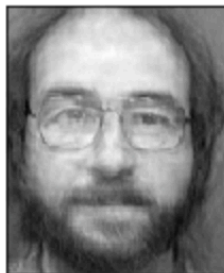
numEigenFaces_80



numEigenFaces_100



numEigenFaces_120



numEigenFaces_140



numEigenFaces_160



numEigenFaces_180



图 3.4: 测试集样例

可以看到，成功重建出了原始的人脸图像，使用的特征脸越多效果越好。

4 实验结果

4.1 样例测试



图 4.5: 样例测试结果

样例测试也成功通过。

5 总结

1. 通过这次实验，对人脸识别的原理有了了解，并能够利用所学知识进行复现。
2. 在获得特征脸的过程中，需要使用主成分分析进行图像压缩，实现降维，有利于减少计算复杂性，并且这里采用了奇异值分解，使得计算成为可能。
3. 基于特征脸，我们可以对输入的图像数据进行压缩，也可以对压缩后的图像进行复原。